

Моделирование и анализ графов атак

Лабораторная работа № 3

Бердыев Эзиз

2026-09-04

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Граф атак	7
3.2	Поиск путей атаки	7
3.3	Метрики центральности	8
3.4	Вероятность успешной атаки	8
4	Выполнение лабораторной работы	10
4.1	Реализация графа атак	10
4.2	Запуск эксперимента	13
4.3	Визуализация и анализ графа	14
4.4	Исследование масштабируемости	17
4.5	Параметрическое исследование	18
4.6	Литературное программирование	21
5	Ответы на контрольные вопросы	22
6	Выводы	24
	Список литературы	26

Список иллюстраций

4.1	Код построения графа атак	10
4.2	Добавление доверительных отношений	11
4.3	Поиск всех путей атаки (DFS с возвратом)	11
4.4	Вычисление метрик центральности	12
4.5	Реализация алгоритма PageRank	12
4.6	Сценарий запуска эксперимента	13
4.7	Результаты выполнения эксперимента	13
4.8	Визуализация графа атак	14
4.9	Анализ графа, часть 1	15
4.10	Анализ графа, часть 2	15
4.11	Сценарий анализа графа	16
4.12	Сценарий исследования масштабируемости	17
4.13	Рост числа путей атаки с увеличением размера сети (логарифмическая шкала)	18
4.14	Сценарий параметрического исследования	19
4.15	Рост числа путей атаки с увеличением плотности сети	20
4.16	Доля сетей, в которых цель достижима из точки входа	20

Список таблиц

3.1	Метрики центральности и их интерпретация в задачах защиты . . .	8
4.1	Результаты основного эксперимента	13
4.2	Критические узлы сети по метрикам центральности	16
4.3	Зависимость числа путей и времени поиска от размера сети	17
4.4	Влияние плотности сети на число путей атаки и достижимость цели	19

1 Цель работы

Целью лабораторной работы является изучение методов построения и анализа графов атак для оценки защищённости сетевой инфраструктуры [1].

В рамках работы требуется не только реализовать алгоритмы, но и понять, каким образом графовые модели позволяют формализовать процесс проникновения, выявлять потенциальные сценарии атаки и находить критические узлы сети.

2 Задание

В соответствии с методическими указаниями [1] требуется:

1. Создать рабочий каталог для кода и установить необходимые пакеты.
2. Построить граф атак для заданной сети.
3. Реализовать алгоритм поиска всех путей атаки.
4. Вычислить метрики центральности.
5. Оценить вероятность успешной атаки и найти наиболее вероятный путь.
6. Визуализировать граф.
7. Провести анализ масштабируемости.
8. Выполнить параметрическое исследование.
9. Преобразовать код в литературный стиль и сгенерировать из него чистый код, Jupyter-блокнот и документацию Quarto.

3 Теоретическое введение

3.1 Граф атак

Граф атак — ориентированный граф $G = (V, E)$, в котором вершины соответствуют состояниям системы (например, узлам сети), а рёбра — возможным действиям злоумышленника, переводящим систему из одного состояния в другое [2]. Такая модель позволяет формализовать процесс проникновения и систематически анализировать сценарии атак [3].

Помимо случайных рёбер, отражающих потенциальные переходы, в модель вводятся *доверительные отношения* — заранее известные связи между узлами, которыми злоумышленник может воспользоваться гарантированно.

3.2 Поиск путей атаки

Для перечисления сценариев применяется поиск в глубину (DFS), находящий все простые пути от точки входа до цели. Ограничение на повторное посещение вершин исключает циклы.

Принципиальное ограничение метода — вычислительная сложность: число простых путей в графе растёт экспоненциально с числом вершин, поэтому полный перебор применим лишь к небольшим фрагментам инфраструктуры [4]. Это ограничение проверяется экспериментально в разделе о масштабируемости.

3.3 Метрики центральности

Для выявления критических узлов используются метрики, сведённые в табл. 3.1.

Табл. 3.1: Метрики центральности и их интерпретация в задачах защиты

Метрика	Что показывает	Интерпретация для защиты
in-degree	Число входящих рёбер	Сколько способов скомпрометировать узел
betweenness	Доля кратчайших путей, проходящих через узел	Узел-посредник, «бутылочное горлышко»
closeness	Близость узла ко всем остальным	Насколько быстро распространится атака
PageRank	Важность узла с учётом важности его соседей	Приоритет узла при распределении ресурсов

Метрика betweenness введена Фрименом [5], PageRank — Пейджем и Брином [6]. Для ориентированного графа атак PageRank особенно содержателен: он оценивает узел тем выше, чем важнее узлы, из которых в него можно попасть.

3.4 Вероятность успешной атаки

Каждому ребру (i, j) сопоставляется вероятность p_{ij} успешной эксплуатации соответствующей уязвимости; на практике она выводится из оценок CVSS [7]. Вероятность пути равна произведению вероятностей его рёбер:

$$P(\pi) = \prod_{(i,j) \in \pi} p_{ij}. \quad (3.1)$$

Прямая максимизация (ур. 3.1) неудобна. Логарифмирование превращает произведение в сумму, а смена знака — задачу максимизации в задачу минимизации:

$$\arg \max_{\pi} \prod_{(i,j) \in \pi} p_{ij} = \arg \min_{\pi} \sum_{(i,j) \in \pi} (-\ln p_{ij}). \quad (3.2)$$

Так как $-\ln p_{ij} \geq 0$ при $p_{ij} \leq 1$, к задаче (1) применим алгоритм Дейкстры.

4 Выполнение лабораторной работы

4.1 Реализация графа атак

Функция построения графа создаёт случайные рёбра с заданной вероятностью (см. рис. 4.1), после чего добавляет доверительные отношения (см. рис. 4.2).

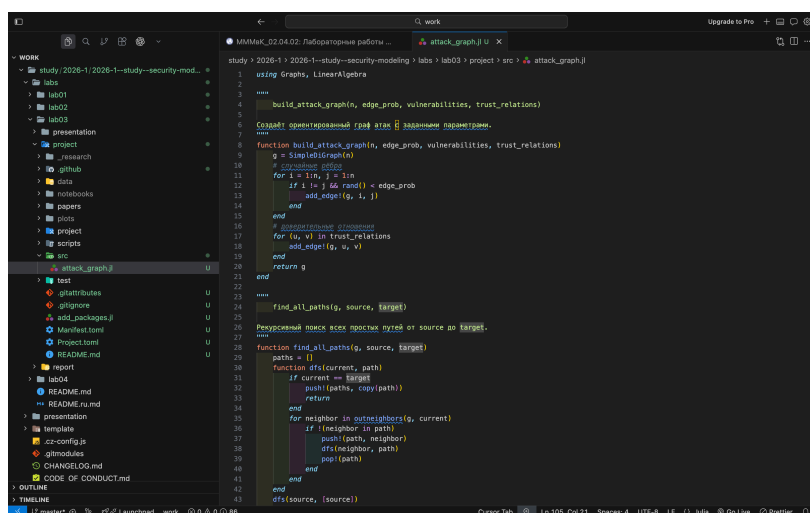


Рис. 4.1: Код построения графа атак

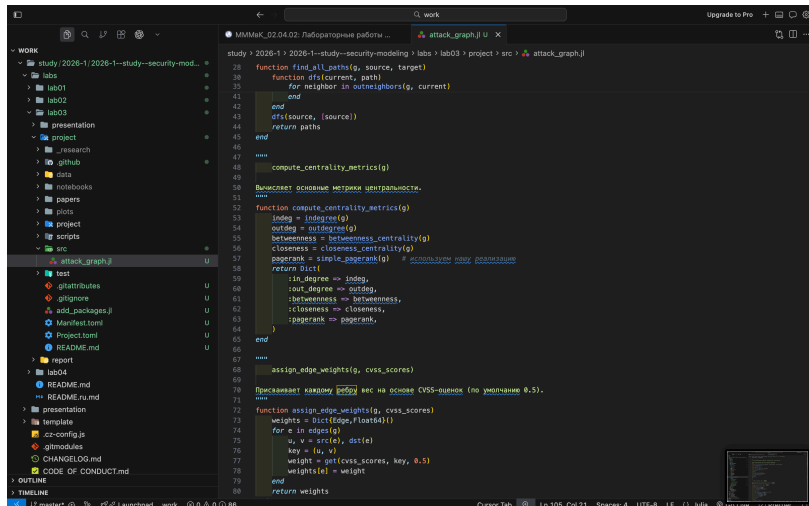


Рис. 4.2: Добавление доверительных отношений

Реализован рекурсивный поиск всех простых путей методом DFS с возвратом (см. рис. 4.3).

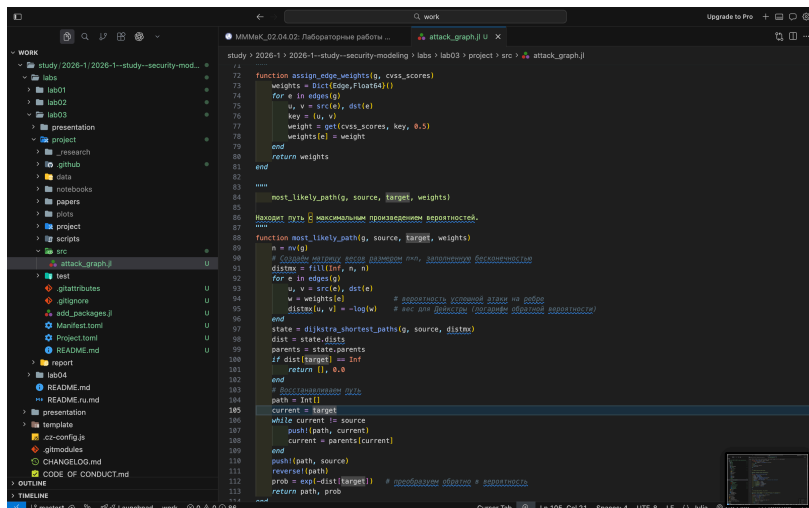


Рис. 4.3: Поиск всех путей атаки (DFS с возвратом)

Вычисление метрик центральности из табл. 3.1 показано на рис. 4.4, собственная реализация PageRank — на рис. 4.5.

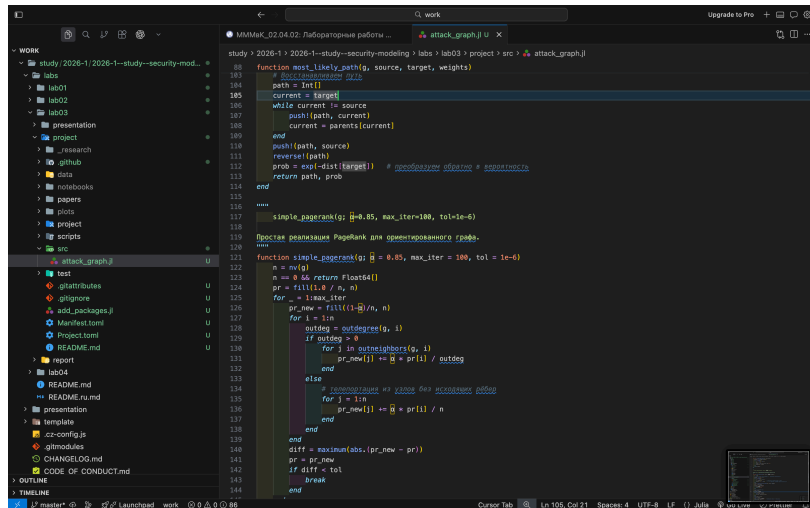


Рис. 4.4: Вычисление метрик центральности

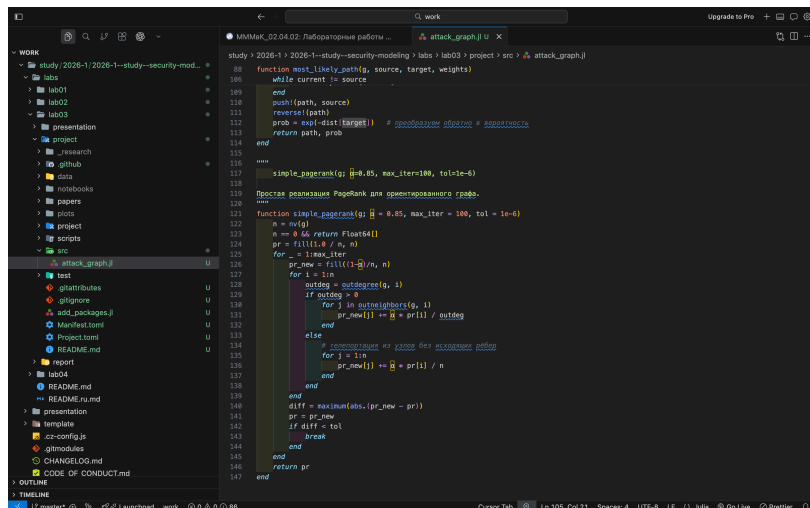


Рис. 4.5: Реализация алгоритма PageRank

В реализации PageRank отдельно обрабатываются узлы без исходящих рёбер: их вес распределяется равномерно по всем вершинам, иначе суммарный вес не сохранился бы.

4.2 Запуск эксперимента

Параметры эксперимента задаются в отдельном сценарии (см. рис. 4.6), результаты выполнения — на рис. 4.7.

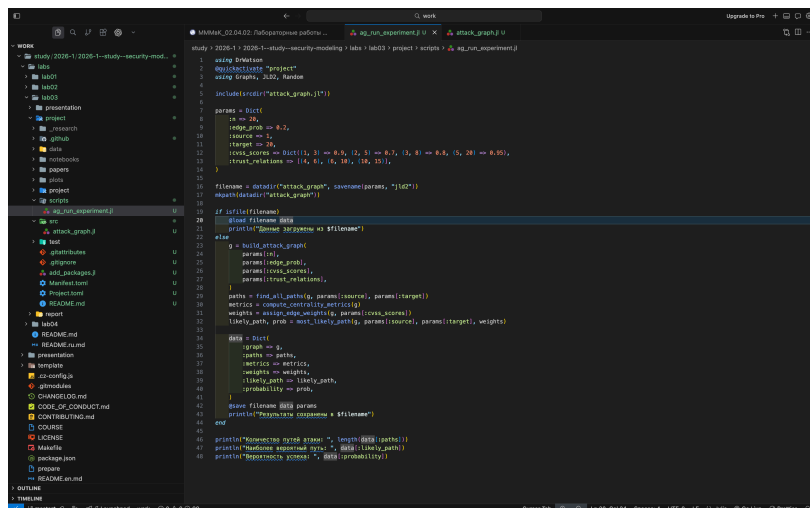


Рис. 4.6: Сценарий запуска эксперимента

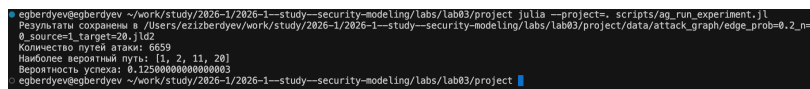


Рис. 4.7: Результаты выполнения эксперимента

Численные характеристики воспроизводимого прогона (зерно генератора случайных чисел зафиксировано значением 20260502) приведены в табл. 4.1.

Табл. 4.1: Результаты основного эксперимента

Характеристика	Значение
Число узлов сети	20
Вероятность ребра	0.2
Число рёбер построенного графа	66
Число простых путей из узла 1 в 20	1814

Характеристика	Значение
Наиболее вероятный путь	$1 \rightarrow 7 \rightarrow 20$
Вероятность наиболее вероятного пути	0.25

Показателен разрыв между 1814 найденными путями и длиной наиболее вероятного из них — всего два перехода. Вероятность $0.25 = 0.5 \times 0.5$ получена на рёбрах со значением по умолчанию: короткий путь оказывается вероятнее длинного, поскольку в произведении (ур. 3.1) каждый дополнительный переход уменьшает итоговую вероятность.

4.3 Визуализация и анализ графа

Построенный граф атак визуализирован с цветовой шкалой по значению метрики центральности (см. рис. 4.8).

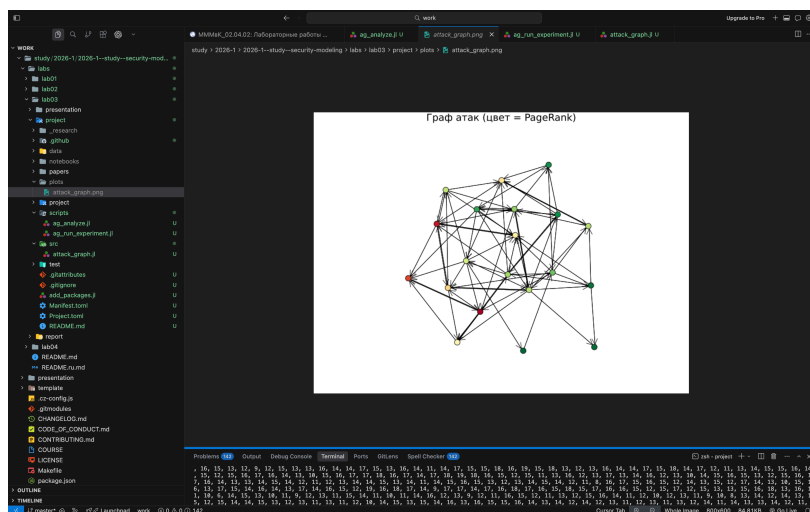


Рис. 4.8: Визуализация графа атак

Результаты анализа графа приведены на рис. 4.9 и рис. 4.10, сценарий анализа — на рис. 4.11.

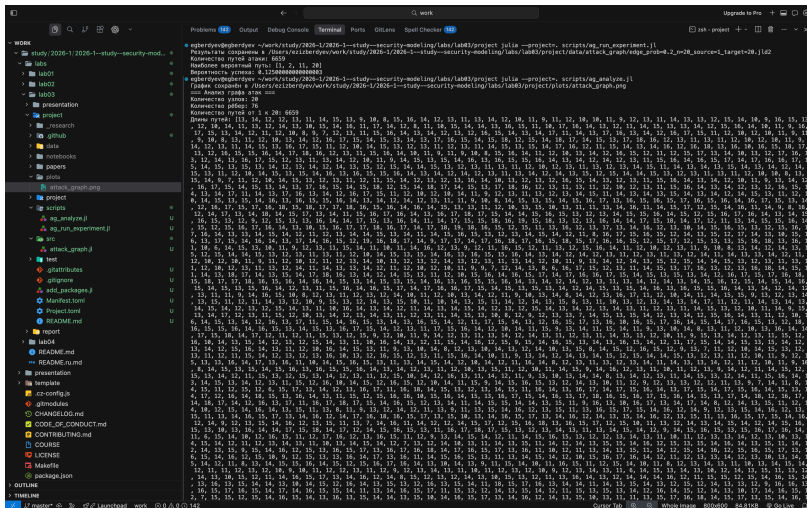


Рис. 4.9: Анализ графа, часть 1

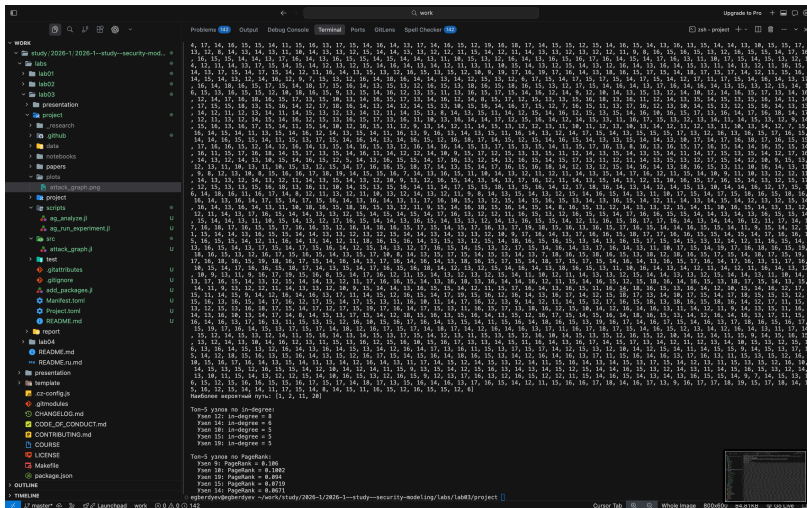


Рис. 4.10: Анализ графа, часть 2

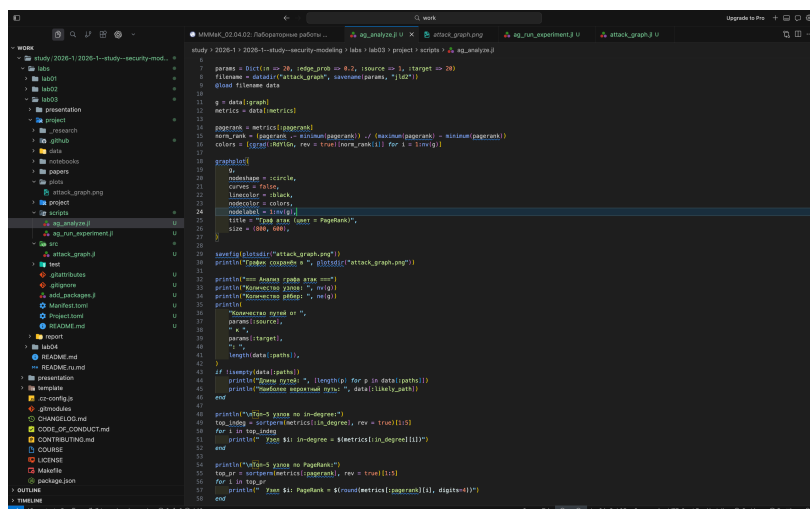


Рис. 4.11: Сценарий анализа графа

Пять наиболее важных узлов по метрике PageRank приведены в табл. 4.2.

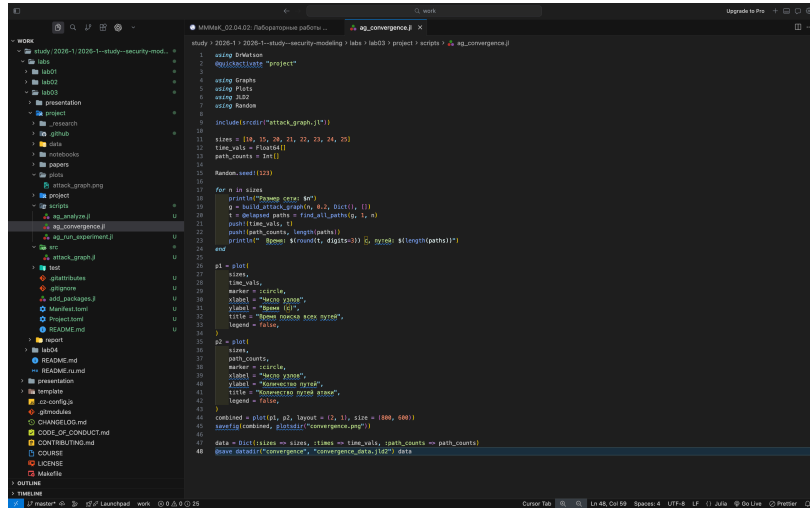
Табл. 4.2: Критические узлы сети по метрикам центральности

Узел	in-degree	betweenness	PageRank
11	7	0.193	0.1484
4	6	0.097	0.1144
6	4	0.304	0.1113
5	7	0.267	0.0985
10	4	0.251	0.0646

Данные табл. 4.2 показывают, что метрики упорядочивают узлы по-разному. Узел 11 лидирует по PageRank, но его betweenness (0.193) ниже, чем у узла 6 (0.304), имеющего вдвое меньший in-degree. Узел 6 входит в число доверительных отношений (4,6) и (6,10) и потому лежит на большом числе путей, оставаясь при этом «незаметным» для метрики степени. Практический вывод: опора на одну метрику даёт неполную картину, критические узлы следует выявлять по совокупности показателей.

4.4 Исследование масштабируемости

Проверяется зависимость числа простых путей и времени их поиска от размера сети (см. рис. 4.12). Отдельный случайный граф — ненадёжная основа для вывода: в нём цель может оказаться недостижимой из точки входа. Поэтому для каждого размера результат усреднён по 20 независимым реализациям графа при фиксированной плотности $p = 0.3$.



```
1 // study / 2025-1 / 2025-1-study-security-modeling / lab1 / lab03 / project / scripts / sq_convergence.cpp
2 #include <iostream>
3 #include <random>
4 #include <igraph>
5 #include <vector>
6 #include <string>
7 #include <algorithm>
8 #include <chrono>
9 #include <cmath>
10 #include <ctime>
11 #include <cstdlib>
12 #include <conio.h>
13 #include <unistd.h>
14 #include <sys/time.h>
15 #include <sys/types.h>
16 #include <sys/stat.h>
17 #include <fcntl.h>
18 #include <syslog.h>
19 #include <syslog.h>
20 #include <syslog.h>
21 #include <syslog.h>
22 #include <syslog.h>
23 #include <syslog.h>
24 #include <syslog.h>
25 #include <syslog.h>
26 #include <syslog.h>
27 #include <syslog.h>
28 #include <syslog.h>
29 #include <syslog.h>
30 #include <syslog.h>
31 #include <syslog.h>
32 #include <syslog.h>
33 #include <syslog.h>
34 #include <syslog.h>
35 #include <syslog.h>
36 #include <syslog.h>
37 #include <syslog.h>
38 #include <syslog.h>
39 #include <syslog.h>
40 #include <syslog.h>
41 #include <syslog.h>
42 #include <syslog.h>
43 #include <syslog.h>
44 #include <syslog.h>
45 #include <syslog.h>
46 #include <syslog.h>
47 #include <syslog.h>
48 #include <syslog.h>
49 #include <syslog.h>
50 #include <syslog.h>
51 #include <syslog.h>
52 #include <syslog.h>
53 #include <syslog.h>
54 #include <syslog.h>
55 #include <syslog.h>
56 #include <syslog.h>
57 #include <syslog.h>
58 #include <syslog.h>
59 #include <syslog.h>
60 #include <syslog.h>
61 #include <syslog.h>
62 #include <syslog.h>
63 #include <syslog.h>
64 #include <syslog.h>
65 #include <syslog.h>
66 #include <syslog.h>
67 #include <syslog.h>
68 #include <syslog.h>
69 #include <syslog.h>
70 #include <syslog.h>
71 #include <syslog.h>
72 #include <syslog.h>
73 #include <syslog.h>
74 #include <syslog.h>
75 #include <syslog.h>
76 #include <syslog.h>
77 #include <syslog.h>
78 #include <syslog.h>
79 #include <syslog.h>
80 #include <syslog.h>
81 #include <syslog.h>
82 #include <syslog.h>
83 #include <syslog.h>
84 #include <syslog.h>
85 #include <syslog.h>
86 #include <syslog.h>
87 #include <syslog.h>
88 #include <syslog.h>
89 #include <syslog.h>
90 #include <syslog.h>
91 #include <syslog.h>
92 #include <syslog.h>
93 #include <syslog.h>
94 #include <syslog.h>
95 #include <syslog.h>
96 #include <syslog.h>
97 #include <syslog.h>
98 #include <syslog.h>
99 #include <syslog.h>
100 #include <syslog.h>
```

Рис. 4.12: Сценарий исследования масштабируемости

Результаты приведены в табл. 4.3 и на рис. 4.13.

Табл. 4.3: Зависимость числа путей и времени поиска от размера сети

Число узлов n	Среднее число путей	Среднее время поиска, с
6	2.15	0.0000007
8	2.85	0.0000008
10	16.05	0.0000026
12	186.50	0.0000331
14	1318.45	0.0002456
16	59932.40	0.0137800

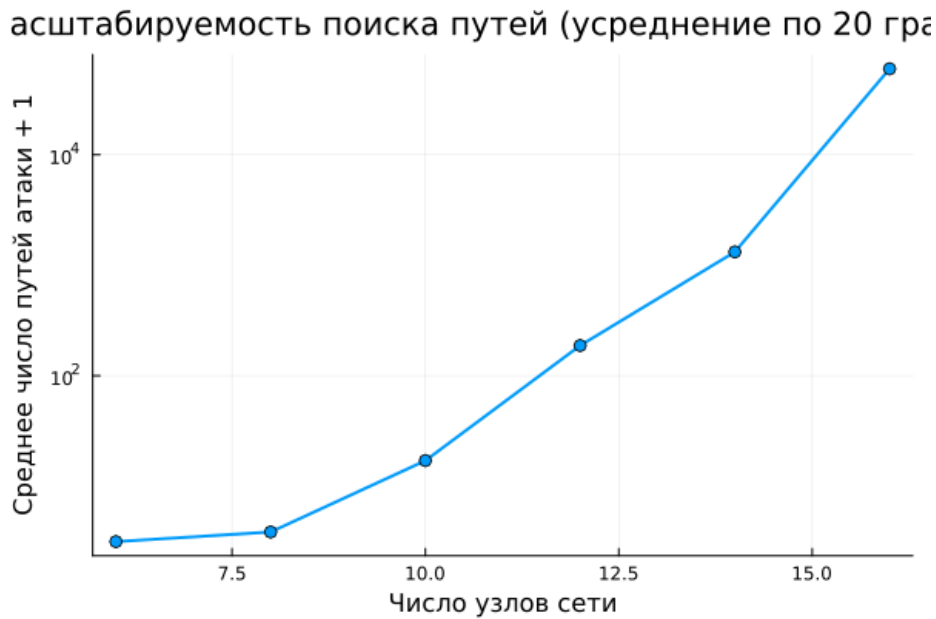


Рис. 4.13: Рост числа путей атаки с увеличением размера сети (логарифмическая шкала)

Рост числа путей носит выраженно экспоненциальный характер: при увеличении числа узлов с 6 до 16, то есть менее чем втрое, среднее число путей возрастает с 2 до 60 тысяч — примерно в 28 тысяч раз. Время поиска растёт пропорционально числу путей: с 0.7 мкс до 13.8 мс, то есть почти в 20 тысяч раз. На логарифмической шкале рис. 4.13 зависимость близка к прямой, что и означает экспоненциальный рост.

Отсюда следует практический вывод: полное перечисление путей атаки нереализуемо уже для сетей в несколько десятков узлов, и для реальной инфраструктуры необходимы метрики центральности, позволяющие выделить критические узлы без построения всех путей.

4.5 Параметрическое исследование

Исследуется влияние плотности сети — вероятности появления ребра — при фиксированном числе узлов $n = 12$ (см. рис. 4.14). Результат также усреднён по 20

реализациям.

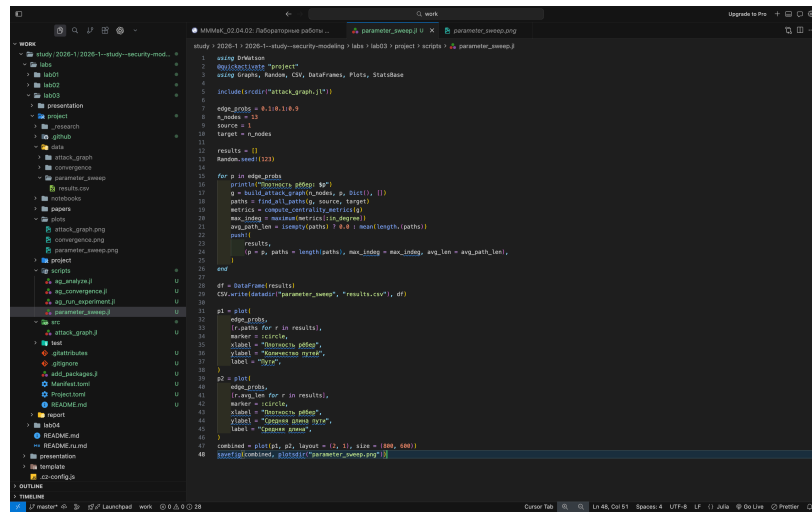


Рис. 4.14: Сценарий параметрического исследования

Результаты приведены в табл. 4.4, графически — на рис. 4.15 и рис. 4.16.

Табл. 4.4: Влияние плотности сети на число путей атаки и достижимость цели

Вероятность ребра	Среднее число рёбер	Среднее число путей	Доля достижимых целей
0.1	12.40	0.30	0.15
0.2	26.45	7.30	0.75
0.3	42.15	358.65	0.95
0.4	53.15	1705.70	1.00
0.5	64.50	9884.30	0.95

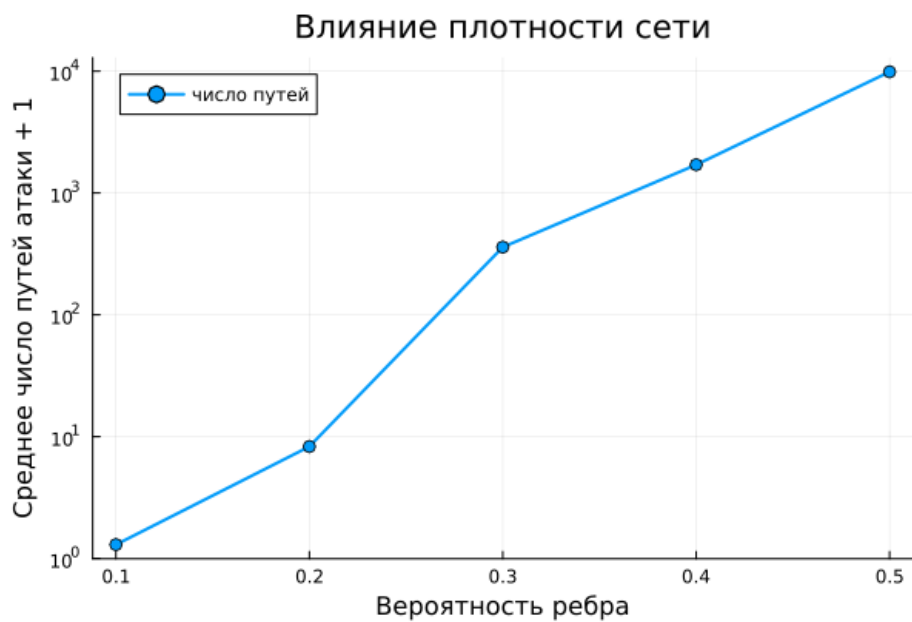


Рис. 4.15: Рост числа путей атаки с увеличением плотности сети

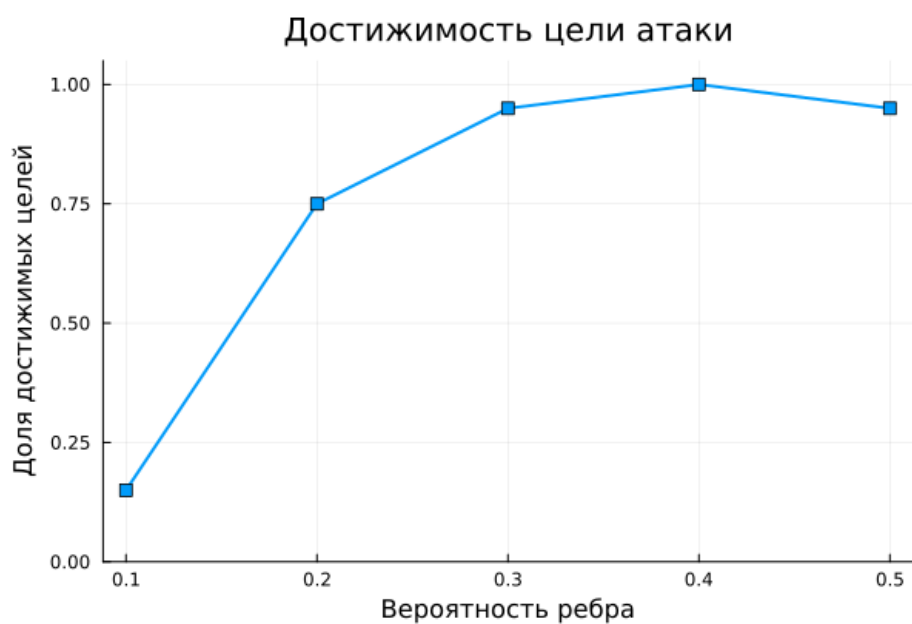


Рис. 4.16: Доля сетей, в которых цель достижима из точки входа

Плотность влияет на результат ещё резче, чем размер: при росте вероятности ребра с 0.1 до 0.5 число рёбер увеличивается вдесятеро, а среднее число путей — более чем в 30 тысяч раз.

Отдельного внимания заслуживает рис. 4.16. Достижимость цели растёт нелинейно и выходит на насыщение уже при $p \approx 0.3$: в 95 % реализаций цель достижима. Снижение до 0.95 при $p = 0.5$ объясняется случайным разбросом при 20 реализациях (одна недостижимая сеть из двадцати) и не является содержательным эффектом. Практический смысл: существует порог связности, после которого дальнейшее «уплотнение» сети уже не меняет самого факта достижимости цели, но продолжает лавинообразно увеличивать число сценариев атаки — то есть усложняет анализ, не меняя принципиального вывода о защищённости.

4.6 Литературное программирование

Код преобразован в литературный стиль средствами Literate.jl. Литературный исходник размещён в каталоге `project/literate`, генерация артефактов выполняется одним сценарием (листинг 4.1).

Листинг 4.1 Генерация артефактов из литературного исходника

```
using Literate

src = projectdir("literate",
                 "20260502T022300--attack-graph__lab03_literate.jl")

Literate.script(src, projectdir("scripts")) # XXXXXX XX
Literate.notebook(src, projectdir("notebooks")) # jupyter notebook
Literate.markdown(src, projectdir("docs")) # XXXXXXXXXXXX Quarto
```

5 Ответы на контрольные вопросы

1. Что такое граф атак?

Ориентированный граф, вершины которого — состояния системы (узлы сети), а рёбра — возможные действия злоумышленника. Модель формализует процесс проникновения и позволяет анализировать сценарии атак [3].

2. Какие алгоритмы используются для анализа?

Поиск в глубину (DFS) — для перечисления всех простых путей атаки; алгоритм Дейкстры — для поиска наиболее вероятного пути после логарифмического преобразования (1).

3. Что показывают метрики центральности?

Каждая метрика отвечает на свой вопрос о важности узла (см. табл. 3.1). Как показано в табл. 4.2, упорядочения по разным метрикам не совпадают, поэтому критические узлы следует выявлять по совокупности показателей.

4. Как оценивается вероятность успешной атаки?

Как произведение вероятностей рёбер вдоль пути (ур. 3.1). Вероятности рёбер выводятся из оценок CVSS [7].

5. Почему для поиска наиболее вероятного пути применяется алгоритм Дейкстры?

Алгоритм Дейкстры минимизирует сумму весов, а не максимизирует произведение. Замена веса ребра на $-\ln p_{ij}$ по формуле (1) сводит одну задачу к другой; веса при этом неотрицательны, что и требуется алгоритмом.

6. В чём ограничения модели?

Модель статична: она не учитывает динамику системы, действия защитника и

изменение состояния узлов после компрометации. Вероятности рёбер считаются независимыми, что на практике выполняется не всегда. Кроме того, полный перебор путей вычислительно неосуществим для больших сетей (см. табл. 4.3).

7. Как можно расширить модель?

Ввести средства защиты как удаление или изменение весов рёбер, учесть стоимость атаки и стоимость защиты, перейти к динамической модели «Защитник—Нападающий», рассматриваемой в следующей лабораторной работе.

6 Выводы

В ходе лабораторной работы:

- реализована модель графа атак с учётом доверительных отношений между узлами;
- реализован поиск всех простых путей атаки методом DFS; в основном эксперименте найдено 1814 путей (см. табл. 4.1);
- вычислены метрики центральности, выявлены критические узлы сети (см. табл. 4.2); показано, что разные метрики дают разные упорядочения и требуют совместного анализа;
- реализована оценка вероятности атаки через произведение вероятностей рёбер и найден наиболее вероятный путь с помощью логарифмического преобразования
(1) и алгоритма Дейкстры;
- выполнена визуализация графа;
- экспериментально подтверждён экспоненциальный рост числа путей атаки: при увеличении сети с 6 до 16 узлов среднее число путей выросло примерно в 28 тысяч раз (см. табл. 4.3);
- проведено параметрическое исследование по плотности сети, выявившее порог связности, после которого достижимость цели выходит на насыщение (см. рис. 4.16);
- код преобразован в литературный стиль, из него сгенерированы чистый код, Jupyter-блокнот и документация Quarto.

Графы атак являются эффективным инструментом анализа защищённости, однако прямое перечисление сценариев неприменимо к реальным сетям. Практическую ценность имеют метрики центральности, позволяющие выделять критические узлы без построения всех путей.

Список литературы

1. Российский университет дружбы народов. Методы математического моделирования в кибербезопасности. 2026.
2. Phillips C., Swiler L. P. A Graph-Based System for Network-Vulnerability Analysis // Proceedings of the 1998 Workshop on New Security Paradigms (NSPW '98). 1998. С. 71–79.
3. Sheyner O. и др. Automated Generation and Analysis of Attack Graphs // Proceedings of the 2002 IEEE Symposium on Security and Privacy. 2002. С. 273–284.
4. Ammann P., Wijesekera D., Kaushik S. Scalable, Graph-Based Network Vulnerability Analysis // Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS '02). 2002. С. 217–224.
5. Freeman L. C. A Set of Measures of Centrality Based on Betweenness // Sociometry. 1977. Т. 40, № 1. С. 35–41.
6. Page L. и др. The PageRank Citation Ranking: Bringing Order to the Web: 1999-66. Stanford InfoLab, 1999.
7. Forum of Incident Response and Security Teams. Common Vulnerability Scoring System version 3.1: Specification Document. 2019.